IntLLM: Compiler-Verified 1.58-Bit Ternary Language Models for Operating-System Kernel Inference

Muhamad Fajar Putranto
TaxPrime / PrimeCore.id, Jakarta, Indonesia

Abstract

We present **IntLLM**, a 1.58-bit ternary language model family (22-74 M parameters) trained from scratch and deployed inside the FajarOS Nova v3.9.0 operating-system kernel. Five contributions: (C1) a three-point scaling chain with monotonically widening calibrated-gate margins ($0.118 \rightarrow 0.279$ nat over a $\sim 3.5\times$ parameter range) extends BitNet-style ternary training validation downward from the literature's 2 B+ regime into the kernel-context- relevant 22-74 M range; (C2) a 25.67 B-token bilingual Indonesian + English corpus (60:40 mix, 0% synthetic) and a Q5 Mini-scale baseline establish the first fully-public bilingual ternary LM resources at this scale, reproducible in $\sim$42 minutes on commodity laptop hardware; (C3) deployment of the Phase D Medium checkpoint via three Fajar Lang `@kernel` modules under a 4-invariant CI regression gate, with binary footprint within the $\leq$ 16 MB design budget at the Mini and Base scales; (C4) two outlier-handling features ported from the post-hoc-quantization literature (per-channel calibration after SmoothQuant; Hadamard rotation after QuaRot) FAIL their mechanical adoption gates by -82.13 and -0.12 nat respectively, with three-cause attributions identifying a common pattern that we hypothesize generalizes: post-hoc-literature precedents do not transfer cleanly to training-from-scratch ternary at small scale; (C5) a four-context Fajar Lang type system enforces the C3 kernel-context invariants at compile time via seven domain-specific error codes. The combination of these five — none individually a breakthrough — is the contribution; we do not claim frontier benchmark performance, FP16 parity at the same parameter count, or formal verification at the seL4 level. The five-entry Phase F research agenda (post-training PTQ, post-hoc QuaRot, and three more) informs by the C4 negative-results and constitutes a more well-posed follow-up than the original five-feature Phase E plan that the empirical evidence has refuted.

1 Introduction

Operating-system kernels are an unusually hostile environment for language models. Heap allocation is forbidden in interrupt-handler context; floating-point hot paths are unreliable across interrupt boundaries on most x86 microkernels; the binary footprint is constrained by physical memory budgets that pre-date the gigabyte-scale weights of modern transformer LMs by decades. Yet there is a growing class of deployment scenarios — drone flight controllers that must classify sensor anomalies in real time, automotive electronic control units that must reason about driver-intent ambiguity inside the safety boundary, embedded sensor hubs that cannot afford userspace context-switch latency for every classification — where the model SHOULD be in the kernel address space rather than shipping data across the user/kernel boundary on every inference.

Existing 1-bit and 1.58-bit ternary LMs come close but do not fit. BitNet b1.58 2B4T [Ma et al., 2024] is $\approx$ 500 MB even after ternary packing, two orders of magnitude over a typical embedded-kernel binary budget. MatMul-Free LLM [Zhu et al., 2024] validates the architecture down to 370 M parameters, $\approx$ 70 MB packed — still too large for the kernel budget but a useful starting point. The eBPF-AI line of work puts inference *near* the kernel via syscall-side hooks, but keeps the model weights in a userspace process that the kernel calls into; the inference still crosses the user/kernel boundary on every call. Standard FP16 transformer deployments (Llama-style, SmolLM2 [Allal et al., 2024], Pythia [Biderman et al., 2023]) are at minimum gigabytes; they do not fit by orders of magnitude.

This paper presents **IntLLM**, a 1.58-bit ternary language model family in the 22-74 M parameter range, trained from scratch under a calibrated three-point scaling chain, deployed entirely inside the FajarOS Nova v3.9.0 kernel address space, and verified at compile time by a four-context type system. The combination of these properties is the contribution; no single property is a breakthrough in isola-

tion, and we do not claim frontier benchmark performance at this scale. What IntLLM *does* establish is that the joint deployment-target intersection of (i) ternary precision, (ii) 22-74 M scale, (iii) bilingual native training, (iv) kernel-resident inference, and (v) compile-time-verified safety is achievable by a single coherent system, on commodity laptop hardware, with all artifacts publicly reproducible.

The paper makes five contributions:

- **C1 (Section 5).** A three-point 1.58-bit ternary scaling chain at 22 M / 46 M / 74 M parameters with monotonically widening calibrated-gate margins (0.118 $\rightarrow$ 0.210 $\rightarrow$ 0.279 nat) over a $\sim$3.5$\times$ parameter range, validating that BitNet-style training-from-scratch ternary extends downward from the literature's 2 B+ regime [Ma et al., 2024] into the parameter range relevant for kernel-context deployment.

- **C2 (Section 6).** A 25.67 B-token fully-public bilingual Indonesian + English corpus at 60:40 mix with 0% synthetic content and a mechanical-consistency-gated reproducibility artifact, plus a Q5 Mini-scale baseline measurement that establishes the cross-lingual coherence floor every subsequent ablation in the paper compares against.

- **C3 (Section 8).** Deployment of the Phase D Medium checkpoint inside the FajarOS Nova v3.9.0 kernel address space via a three-module Fajar Lang `@kernel`-context implementation, gated by a 4-invariant CI regression test, with binary footprint within the $\leq$ 16 MB design budget at the Mini and Base scales.

- **C4 (Section 7).** Two outlier-handling features ported from the post-hoc-quantization literature [Xiao et al., 2022, Ashkboos et al., 2024] ablated against the C2 Q5 baseline both FAIL their mechanical adoption gates (-82.13 and -0.12 nat respectively), with three-cause attributions identifying a common failure mode that we hypothesize generalizes beyond the two specific features tested: *post-hoc-literature precedents do not transfer cleanly to training-from-scratch ternary at small scale.*

- **C5 (Section 9).** A four-context type system (`@safe`, `@kernel`, `@device`, `@unsafe`) implemented in the Fajar Lang compiler that enforces the C3 kernel-context invariants at compile time via seven domain-specific error codes (KE001-KE004, DE001-DE003), positioned on the safety-mechanism spectrum stronger than userspace-ML documentation-grade conventions and weaker (explicitly) than seL4-class machine-checked proofs [Klein et al., 2009].

C1-C3 are *deployment positives* establishing that the IntLLM family is a working artifact at the stated intersection. C4 is a *methodological negative* — two ablations that FAIL — published honestly per the project's research-integrity rules (Section 4.5) because the failures inform a five-entry Phase F follow-up agenda (Section 10.3) more usefully than "we tried something and it didn't work" would. C5 ties C3 to the compiler infrastructure, lifting kernel-context safety from review-time convention to compile-time guarantee.

1.1 What this paper is not

The paper does not claim:

- competitive multi-choice reasoning benchmark scores, nor parity with FP16 baselines at the same parameter count — the 22-74 M scale is below the emergence threshold for those skills per Biderman et al. [2023] and the trade for ternary precision is explicit;

- that the C4 outlier-handling failures imply universal failure of the QuaRot/SmoothQuant family — Phase F.6 explicitly tests whether the post-hoc protocol recovers a positive result on the trained checkpoint;

- that FajarOS Nova or the Fajar Lang analyzer are formally verified or production-hardened — both are research-grade, and Section 9.5 delineates the gap to seL4 [Klein et al., 2009]-class guarantees explicitly;

- bare-metal performance numbers (tok/s, watts) — Table 10 tags those entries PEND, deferred to a future deployment-platform partner.

The paper organization follows the C1-C5 contribution stack, with Sections 2-4 establishing background and methodology and Section 10 closing with the Phase F follow-up agenda.

2 Background

This section establishes the prior work that IntLLM extends along three axes: ternary language modeling (Section 2.1), kernel-context deployment constraints (Section 2.2), and the Fajar Lang context-annotation system that the architecture of Section 3 uses without further explanation (Section 2.3). Readers familiar with one or more of these axes may skip directly to Section 3; the cross-reference graph is self-contained.

2.1 Ternary language models

The lineage of weight-quantized language models that IntLLM extends runs from Wang et al. [2023]'s BitNet through Ma et al. [2024]'s BitNet b1.58 to Zhu et al. [2024]'s MatMul-Free LLM (MMfreeLM). The line of evolution is straightforward to summarize at a high level:

BitNet [Wang et al., 2023] introduces `BitLinear`, a drop-in replacement for `nn.Linear` that quantizes weights to $\{-1, +1\}$ during pretraining via the straight-through estimator (STE) and quantizes activations to 8-bit per-token absmax. Validates that 1-bit weights can match FP16 baselines at ≥ 700 M parameters.

BitNet b1.58 [Ma et al., 2024] extends the weight quantization grid from $\{-1, +1\}$ to $\{-1, 0, +1\}$ (ternary), recovering one extra dynamic-range degree of freedom for an effective $\log_2 3 \approx 1.58$ bits per weight. Reports parity with full-precision baselines on standard zero-shot benchmarks at the 2 B-parameter scale (BitNet b1.58 2B4T release on HuggingFace as `microsoft/bitnet-b1.58-2B-4T`). The ternary recipe defines the absmean weight scale $s_w = 1/\overline{|W|}$ that we adopt verbatim in Equation 6.

MatMul-Free LLM [Zhu et al., 2024] structurally eliminates the matmul-heavy softmax-attention block by replacing it with a gated linear recurrence (HGRN — Hierarchically Gated Recurrent Network) that uses four BitLinear projections per attention block (`i_proj`, `f_proj`, `g_proj`, `o_proj`). The MLP block reduces from three FP matmuls to two BitLinear projections (`gate_proj`, `down_proj`) plus an element-wise gate. Result: the only matmul in the forward path is the output projection to vocabulary (`lm_head`), and even that is BitLinear; the forward path has zero FP matmuls in the inference hot path. Validates the architecture at 370 M, 1.3 B, and 2.7 B parameters; ships an FPGA accelerator at 13 W.

The literature gap that IntLLM closes is downward in scale: BitNet b1.58 validates at 2 B; MMfreeLM at ≥ 370 M. Neither validates at the 22-74 M parameter range that is binding for kernel-context deployment. Whether the BitNet/MMfreeLM scaling-law assumptions extend down to that range is an empirical question we address in Section 5.

A parallel post-hoc-quantization line of research (SmoothQuant [Xiao et al., 2022], GPTQ [Frantar et al., 2022], AWQ [Lin et al., 2023], QuaRot [Ashkboos et al., 2024], SpinQuant [Liu et al., 2024]) compresses pre-trained large transformers to 4-8 bit weights without retraining. We do not compete with this line directly: the IntLLM problem is to TRAIN ternary from scratch at small scale, not to compress a pre-trained large model. We do draw on this line for outlier-handling features (Section 7); the empirical observation that those features fail in our regime is contribution C4.

2.2 Kernel-context deployment constraints

Kernel-context deployment of language models is not just userspace deployment with a smaller memory budget; the constraints are structurally different in three ways that bear on every architectural choice in this paper:

1. **No heap allocation during inference.** Most kernel schedulers explicitly forbid heap allocation in interrupt-handler context (a deadlock would result if the heap allocator's lock is already held by the interrupted code) and discourage it in non-interrupt kernel context for predictability reasons. This rules out PyTorch-style dynamic tensor allocation, dynamic-vocabulary tokenizers, and any inference path that constructs new objects per forward pass. IntLLM's `@kernel` forward path uses only statically-allocated buffers sized at boot time.

2. **Floating-point hot path unreliability.** Most x86 microkernels save the FPU/SSE/AVX register state lazily on context switches; gratuitous FP arithmetic in interrupt context can corrupt the userspace process's FP state. Convention on Linux and most microkernels is to either (a) explicitly save/restore FP state around any kernel-side FP arithmetic (with significant per-call cost) or (b) simply not do FP arithmetic in kernel hot paths. IntLLM's `@kernel` forward path uses i64 fixed-point arithmetic throughout; the only FP-stored values are RMSNorm β scales and 257-entry sigmoid/SiLU lookup-table endpoints, both decoded to fixed-point at `model-load` time and never multiplied as FP at inference (Section 3.6).

3. **Binary footprint constraints.** Kernel-resident binary budgets pre-date the gigabyte-scale weights of modern transformer LMs by decades. Typical embedded-kernel images target ≤ 4 MB (seL4 reference [Klein et al., 2009], 6 kLoC); ≤ 16 MB is generous. The IntLLM ≤ 16 MB design budget targets "can fit on commodity ARM-class kernel-image-friendly hardware" rather than "smallest possible"; even at this generous budget BitNet

b1.58 2B4T ($\sim$ 500 MB packed) is two orders of magnitude over.

The eBPF / eBPF-AI line of work [Starovoitov et al., 2014] bridges some of these constraints by allowing limited userspace-loaded bytecode to run inside kernel-event handlers, but the model weights themselves typically remain in a userspace process that the eBPF program calls into. The user/kernel transition cost on every inference is what our `@kernel` resident model avoids.

2.3 Fajar Lang `@kernel` annotation system

The IntLLM kernel-path implementation uses Fajar Lang's four-context type system to enforce the constraints of Section 2.2 at compile time. We give a preview here so that the annotations in Section 3's code listings are self-explanatory; the type system itself is the subject of contribution C5 (Section 9).

Every Fajar Lang function is annotated with one of four contexts:

- `@safe` (default): userspace business logic, no hardware access, no raw pointers.
- `@kernel`: kernel-resident code; allowed stack allocation and hardware-register intrinsics, forbidden heap allocation, tensor construction, and `@device` function calls.
- `@device`: tensor-runtime / accelerator code; allowed tensor operations, forbidden raw pointers and `@kernel` function calls.
- `@unsafe`: escape hatch for FFI shims, audited block-by-block.

Calls that would cross domain boundaries inappropriately are mechanically rejected at compile time with one of the seven KE001-KE004 / DE001-DE003 error codes (Table 12). The IntLLM kernel-path orchestrator `tfm_matmulfree.fj`, the kernel ops module `matmulfree.fj`, and the model-format parser `fjm_v9.fj` are all `@kernel`-annotated; the bridge from kernel-side I/O to device-side tensor construction (when used outside kernel deployment, e.g. during training) goes through `@safe` glue functions following the pattern of Section 9.4. Readers can treat the `@kernel` annotations in Section 3 as "this function will compile to a kernel-resident binary, and the analyzer has verified it does not violate the constraints of Section 2.2."

3 IntLLM Architecture

IntLLM is a decoder-only language model family that extends the MatMul-Free LLM architecture [Zhu et al., 2024] with full integer-native operation, kernel-context-friendly memory layout, and a calibrated ternary quantization pipeline that runs end-to-end inside the FajarOS Nova `@kernel` address space.

3.1 Model class hierarchy

The training-time class hierarchy is `HGRNBitForCausalLM` $\rightarrow$ `HGRNBitModel` $\rightarrow$ `HGRNBitBlock`, inherited verbatim from the upstream MatMul-Free LLM repository (frozen pin in `python/phase_d/_upstream/mmfreelm/`). IntLLM extends the upstream surface in three places without modifying upstream sources: (i) a custom training driver `intllm.train.train_loop` that adds calibrated gates per scale, checkpoint atomicity, and a step-idle watchdog (per the cross-repo production-safety rule of Section 4.3); (ii) a quantization helper module `intllm.quant` adding RMSNorm variants, sigmoid/SiLU lookup tables, and the per-channel adaptive quantizer (Section 3.2); and (iii) an evaluation harness `intllm.eval` aligning measurements with `lm-evaluation-harness v0.4.11` for baseline parity per the research-integrity rules of Section 4.5.

3.2 BitLinear forward path

The fundamental quantized operator is `BitLinear` [Wang et al., 2023, Ma et al., 2024], a drop-in replacement for `nn.Linear` that quantizes both weights and activations during training. The forward path is

$$y = \mathrm{F.linear}\big(\widehat{x}_{\mathrm{quant}},\ \widehat{W}_{\mathrm{quant}}\big), \tag{1}$$

where the activation and weight quantizers, with straight-through estimator (STE) gradient passthrough, are

$$x_{\mathrm{norm}} = \mathrm{RMSNorm}(x), \tag{2}$$

$$\widehat{x}_{\mathrm{quant}} = x_{\mathrm{norm}} + \mathrm{StopGrad}\big(q_a(x_{\mathrm{norm}}) - x_{\mathrm{norm}}\big), \tag{3}$$

$$\widehat{W}_{\mathrm{quant}} = W + \mathrm{StopGrad}\big(q_w(W) - W\big). \tag{4}$$

The activation quantizer q_a is per-token absmax 8-bit:

$$q_a(x)_{t,c} = \frac{\lfloor x_{t,c} \cdot s_t \rceil_{[-128,127]}}{s_t}, \quad s_t = \frac{127}{\max_{c'} |x_{t,c'}| + \epsilon}, \tag{5}$$

where $\lfloor \cdot \rceil_{[-128,127]}$ denotes round-then-clamp. The weight quantizer q_w is per-tensor absmean ternary $\{-1, 0, +1\}$ per b1.58:

$$q_w(W)_{i,j} = \frac{\lfloor W_{i,j} \cdot s_w \rceil_{[-1,1]}}{s_w}, \quad s_w = \frac{1}{\overline{|W|} + \epsilon}, \quad (6)$$

giving an effective $\log_2 3 \approx 1.58$ bits per weight after entropy coding. We use $\epsilon = 10^{-5}$ throughout, matching the upstream implementation.

3.3 Per-block layout

Each `HGRNBitBlock` contains exactly six BitLinear instances: four in the attention sub-layer (`i_proj`, `f_proj`, `g_proj`, `o_proj`) implementing the gated linear recurrence of MMfreeLM [Zhu et al., 2024], and two in the feed-forward sub-layer (`mlp.gate_proj`, `mlp.down_proj`). A top-level `lm_head` BitLinear projects to the vocabulary, giving a per-model BitLinear count of $6L + 1$, where L is the number of layers. For our Mini configuration ($L = 6$), this is 37 BitLinear sites; this count is recoverable as a runtime invariant via `intllm.qat.is_bitlinear`-based traversal and is reported by every Phase D ablation harness.

The structural distinction between `i_proj`/`f_proj`/`g_proj` and `o_proj` is load-bearing for our negative-result analysis in Section 7: the first three implement gated linear recurrence projections (structurally distinct from the QKV projections of softmax attention) while `o_proj` is the post-recurrence output projection. Outlier-handling rotations from the QuaRot family [Ashkboos et al., 2024, Liu et al., 2024] are canonically applied to QKV inputs in transformers; we test the closest analogue (`o_proj` input rotation) at Mini scale and find it insufficient (Section 7).

3.4 Tokenizer and vocabulary

We use the Mistral v3 byte-pair encoding tokenizer with 32,768 token vocabulary, chosen because (i) it has wide multilingual coverage at a size that fits the kernel-binary budget, (ii) it is the same tokenizer used by Zhu et al. [2024] so cross-paper benchmarks are directly comparable, and (iii) its training data spans Latin-script languages including Indonesian, supporting the bilingual extension of Section 6 without retraining the tokenizer.

3.5 Configuration matrix

Three IntLLM scales validate the Phase D scaling-chain contribution (C1, Section 5). All three share the BitLinear forward path of Section 3.2 and the per-block layout of Section 3.3; they differ only in hidden dimension and depth.

Table 1: IntLLM configuration matrix. Parameter counts include the ternary BitLinear weights, FP32 norms, and the embedding+lm_head matrices at full precision in the trained checkpoint. The `.fjm` v9 export format reduces the trained checkpoint to a ternary-packed kernel binary.

Config	d	L	Vocab	Params
Mini	256	6	32,768	21.9 M
Base	384	12	32,768	46.4 M
Medium	512	12	32,768	74.5 M

The hidden dimensions $\{256, 384, 512\}$ are deliberately chosen as powers-of-two-friendly values: $d = 2^k$ supports the Walsh-Hadamard rotation construction of Section 7 without padding, $d \cdot 4/3$ MLP intermediate sizes pack cleanly into ternary kernel buffers, and the embedding tables fit in $\leq 2^{15}$ vocabulary without dynamic sizing.

3.6 Integer-native activation pipeline

Beyond BitLinear, three additional surface-area changes make the forward path integer-native end-to-end:

γ-less RMSNorm. The upstream MatMul-Free RMSNorm multiplies its normalized output by a learnable γ tensor; we absorb this scale into the downstream BitLinear's effective β calibration constant during checkpoint export, eliminating a per-block FP multiplication from the kernel-resident forward path. No training-time loss-of-expressivity penalty was observed in the Phase D scaling chain (Section 5).

Sigmoid and SiLU lookup tables. The HGRN gating activation σ and the MLP gate activation SiLU are replaced at inference time by 257-entry linear-interpolation lookup tables defined over $x \in [-8, +8]$, with maximum absolute error $\leq 2 \cdot 10^{-3}$ and $\leq 1.6 \cdot 10^{-2}$ respectively. The training-time forward path uses standard PyTorch `torch.sigmoid` and `torch.nn.functional.silu` for gradient stability; the training-versus-inference equivalence is verified by an `fp16-vs-ternary parity gate` (deferred to Path A Week 3 calendar; Section 4.3).

Activation quantization at module boundary. The activation quantizer q_a of Equation 5 is applied

Table 2: Architecture comparison on axes relevant to kernel-context deployment. "✓" = present, "○" = partial, "–" = absent. The integer-native column requires zero floating-point matmuls in the forward path.

	Tern. wts	Int. acts	No softmax	No FP at Kernel
BitNet b1.58 2B4T	✓	–	–	–
MatMul-Free 2.7B	✓	–	✓	–
TerEffic FPGA	✓	○	–	✓
IntLLM	✓	✓	✓	✓

at every BitLinear input, producing 8-bit integer activations that are multiplied by ternary weights and accumulated into 16-bit (training) or 32-bit (inference) integers before dequantization. At no point in the forward path does the kernel-resident inference code path require a floating-point matmul, satisfying the kernel-context invariant of zero-FP-hot-path execution.

3.7 Comparison with prior architectures

Table 2 positions IntLLM against the closest prior architectures along the axes that drive the kernel-deployment constraint (full citations in `docs/INTLLM_RELATED_WORK.md` §7).

The composition of the five ✓'s in IntLLM's row, rather than any single axis, is the architectural contribution. We do not claim that any axis is novel in isolation; we claim that the joint satisfaction of all five is what unlocks the kernel-resident deployment demonstrated in Section 8.

4 Training Methodology and Track B Interruption Safety

The IntLLM training pipeline is structured around five methodological commitments that together support the C1 scaling-chain and C4 negative-results contributions: calibrated per-scale val_loss gates (§4.1), a bilingual training stream matching the Q5 baseline protocol (§4.2), a six-layer interruption-safety harness (§4.3), a like-for-like 24K-step ablation budget (§4.4), and a set of research-integrity rules adopted from the project's `CLAUDE.md §6.9` ruleset (§4.5).

4.1 Calibrated per-scale val_loss gates

Each IntLLM configuration ships with a single mechanical pass/fail threshold on held-out validation loss, against which the trained checkpoint is judged. The thresholds are **Mini** < 4.5, **Base** < 4.2, and **Medium** < 4.0 for the SlimPajama-derived English validation stream (50 batches at `seed=999`). The thresholds are *calibrated* against empirical Phase D evidence rather than chosen *a priori*: an earlier Mini v1 run used the uncalibrated target < 4.0 and FAILED (val_loss = 4.46); the same checkpoint, re-evaluated against the calibrated Mini target < 4.5, PASSES with margin 0.04 nat. Mini v2 widens that margin to 0.12 nat, Base c.1 to 0.21 nat, and Medium c.1 to 0.28 nat. The *monotonic widening* of margins across scales is the scaling-chain claim of contribution C1 (Section 5); we treat it as the principal structural signal in the paper rather than any single absolute val_loss number, because absolute losses are sensitive to tokenizer choice and validation-stream sampling that other ternary papers do not hold fixed.

Each gate decision is committed as a separate decision document (`FJQ_PHASE_D_MINI_GATE.md`, `_BASE_GATE.md`, and `_MEDIUM_GATE.md`) per the "decisions are mechanical, not paragraphs" rule of Section 4.5, R6.

4.2 Bilingual training stream

For the bilingual extension (contribution C2, Section 6), the Phase E1 corpus v1.0 (25.67 B tokens, 60:40 ID:EN, 0% synthetic, 0.0254% exact-hash dedup) feeds an interleaver `intllm.data.bilingual_stream(id_share=0.6)` that mixes the Indonesian parquet shards and a streaming SlimPajama English source at per-batch granularity via a seeded coin flip. Distinct random seeds are used for the Indonesian source (`seed`), English source (`seed+1`), and the per-batch language-selection RNG (`seed+2`) so that the coin-flip cannot accidentally correlate with shard ordering on either side.

The *Q5 baseline protocol* that all subsequent E2 ablations compare against is a Mini-scale training run on this bilingual stream for 24,000 steps at batch size 8 and sequence length 1024, yielding $\approx$ 196 M tokens of training data at the design 60:40 ratio. The protocol is implemented in `python/phase_d/scripts/q5_bilingual_baseline.py` and produces the artifact `paper/intllm/ablations/q5_bilingual_baseline.json`, against which all E2.x ablation rows are measured.

4.3 Track B six-layer interruption safety

A Phase D Base c.1 training run on 2026-04-22 hung for 8.5 hours after a laptop suspension event killed the HuggingFace CDN socket beneath the streaming dataloader. The training loop continued executing but produced zero step progress; no checkpoint had been saved and no watchdog detected the hang. The incident motivated a six-layer interruption-safety rule that we codify here for any training script in the IntLLM / FajarQuant tree:

1. **Periodic atomic checkpointing.** Every N steps (N = 10,000 for the Phase D Mini/Base/Medium runs), the model, optimizer, and LR-scheduler states are serialized via `torch.save` to `ckpt_step_{step:06d}.pt.tmp`, then renamed via `os.replace`, which is POSIX-atomic on the same filesystem. The next-write-wins property of `os.replace` guarantees no half-written checkpoint is observed by a resuming reader, even under SIGKILL during the write.

2. **Bit-exact resume.** A `-resume <path>` CLI flag loads the model state-dict, optimizer state, and LR-scheduler state from a checkpoint and continues training with the step counter restored. The unit test `test_train_watchdog.py::test_resume_bit_exact_loss` verifies that pre-save and post-resume one-step losses match within 10^{-4}.

3. **Step-idle watchdog.** A daemon thread polls the step counter every 30 seconds; if the counter has not advanced for 1,800 seconds (30 minutes), it sends `SIGTERM` to the main process. Combined with layer 1 and layer 2 above, the operator can restart the training script with `-resume-auto` and lose at most $\sim 1.5 \times$ `ckpt_every` steps of compute per incident.

4. **Per-chunk read timeouts on streaming sources.** The environment variables `HF_DATASETS_DOWNLOAD_TIMEOUT=60` and `HF_HUB_DOWNLOAD_TIMEOUT=60` are set on import of `intllm.data`, capping any single HTTP chunk read at 60 s; a dead socket raises a timeout instead of hanging indefinitely.

5. **Streaming retry on transient network errors.** A wrapper `intllm.data._retry_iter` reconstructs the streaming iterator on transient exceptions (`ConnectionError`, `OSError`, `requests.Timeout`, `urllib3.ReadTimeoutError`, et al.), with exponential backoff capped at 60 s and a per-retry seed offset so retries see a different shuffle order. Non-retryable exceptions (`ValueError`, `KeyError`) propagate immediately.

6. **Regression gate.** A Makefile target `make test-train-watchdog` runs $\sim$24 unit tests covering the watchdog real-thread fire path, default `SIGTERM` delivery, checkpoint rotation, `-resume` bit-exact load, and `retry_iter` on a synthetic transient exception. A pre-push hook gates training-code changes on this target's pass.

The harness was validated end-to-end during a manual laptop-shutdown event on 2026-04-23 mid-Medium-training: the watchdog fired, the checkpoint was preserved, `-resume-auto` bit-exact-restored the model, and only $\sim$36 minutes of compute was lost out of a $\sim$10 hour run. Subsequent Phase E2 ablations (Sections 6 and 7) used the same harness without modification.

4.4 Like-for-like ablation budget

Every E2.x ablation in the present paper trains for exactly the same 24,000-step budget at the same batch size and sequence length as the Q5 baseline of Section 4.2, i.e. a constant $\approx$196 M-token training-token budget. The budget is exposed as the constant `E2_DEFAULT_N_STEPS=24_000` at the top of the ablation harness driver `train_mini_ablation.py`; an earlier draft of the driver defaulted to the larger 60,000-step Phase D Mini target, which was caught and fixed during the Hadamard ablation launch (commit `ed5a7c2`) when it was noticed that an EN-only 60,000-step run was not directly comparable to the bilingual 24,000-step Q5 baseline.

The orthogonal flag `-bilingual-data` switches the training stream from EN-only SlimPajama to the bilingual mix described in Section 4.2, so any E2 feature flag (`-hadamard`, `-balanced-calib`, etc.) can be ablated against either the EN-only Phase D Mini-gate baseline or the bilingual Q5 baseline. The negative-results experiments of Section 7 use `-bilingual-data` throughout because that is the regime the paper's bilingual contribution lives in.

The principal val_loss comparison metric across all E2 rows is `val_loss(EN, seed=999, 50 batches)`, identical to the Q5 baseline's English measurement, so each ablation reports a single delta against a single baseline rather than a vector of weakly comparable numbers.

4.5 Research-integrity rules adopted from CLAUDE.md §6.9

The methodology section closes with the seven research-integrity rules adopted verbatim from the project's `CLAUDE.md §6.9` ruleset, since those rules materially constrain how each contribution in this paper was produced:

R1 Canonical-protocol benchmarks. All comparison benchmarks use `lm-evaluation-harness v0.4.11` with no custom protocol deviation. The harness SHA is pinned in `eval/HARNESS_SHA` and CI fails on drift.

R2 Literature review precedes algorithm design. The related-work synthesis (`docs/INTLLM_RELATED_WORK.md`, 18 papers across 7 clusters) was committed before the results sections of this paper were drafted, satisfying the "literature first" invariant.

R3 Baseline parity (port full features, no strawman). Comparison baselines (BitNet b1.58 2B4T, MMfreeLM 370M/1.3B/2.7B, SmolLM2 135M/360M, Pythia 160M) are evaluated as published; any unported feature is recorded in `docs/BASELINE_UNPORTED_FEATURES.md` so reviewers can adjust their judgments rather than be misled by silent strawman baselines.

R4 Calibrated > per-chunk for data-driven decompositions. The BitLinear weight-quant scale s_w is calibrated once at QAT freeze rather than recomputed per forward pass, eliminating a correlated noise source on the activation-quantization grid.

R5 Outlier-handling required for LLM quantization. Every ablation in Section 7 has an explicit outlier-handling rationale and adoption gate; the negative-result attribution of Section 7 is itself an instance of this rule's mechanical-gate enforcement.

R6 Results section last. The architecture (Section 3) and methodology (this section) were drafted before any results table was populated; results sections in this paper draw exclusively from the JSON artifact directory `paper/intllm/results/` and are auto-generated rather than hand-edited.

R7 Pre-publication audit gate. The script `scripts/verify_intllm_tables.py -strict` is registered as a pre-push hook and a CI required check; every floating-point literal in this manuscript's tables maps to a registered claim backed by a JSON cell, with the script failing on drift between table and source.

Table 3: Phase D scaling-chain gate decisions. *val_loss* is measured on a 50-batch SlimPajama validation stream at `seed=999`. Gate thresholds are calibrated against Phase D empirical evidence per `FJQ_PHASE_D_GATE_CALIBRATION.md`; the per-scale decision documents (`FJQ_PHASE_D_{MINI,BASE,MEDIUM}_C1_GATE.md`) commit each verdict mechanically. *Margin* is gate − val_loss; positive margin = PASS.

Config	Params	Tokens	val_loss	Gate	Margin
Mini	21.5 M	491.5 M	4.382	4.5	**0.118**
Base	46.5 M	983.0 M	3.990	4.2	**0.210**
Medium	74.5 M	1,818.6 M	3.721	4.0	**0.279**

The combination of mechanical gates (R6, R7) and committed-decision artifacts (R5 + the per-scale gate decision documents of Section 4.1) is what gives the paper's negative results their specific weight: the failures of Section 7 cannot be explained away as "we tried something and it didn't work," because the gates that ruled them out were specified *before* the experiments ran and ran *mechanically* on the artifacts rather than through interpretive re-reading after the fact.

5 Phase D Scaling-Chain Results (C1)

The Phase D scaling-chain consists of three IntLLM configurations trained from scratch under the methodology of Section 4: `intllm-mini` (21.5 M params, 491.5 M tokens, 22.86 tok/param), `intllm-base` (46.5 M params, 983.0 M tokens, 21.16 tok/param), and `intllm-medium` (74.5 M params, 1.819 B tokens, 24.40 tok/param). All three use the SlimPajama-6B streaming dataset, the Mistral v3 32K BPE tokenizer, and the calibrated per-scale gates of Section 4.1. Tokens-per-parameter ratios are within ±15% of the Chinchilla compute-optimal regime [Hoffmann et al., 2022].

5.1 Three-point monotonic gate-margin widening

The scaling-chain claim of contribution C1 is structural: *the gate margins widen monotonically as the model scales up,* indicating that the additional parameters and training tokens go into reducing val_loss faster than the gate threshold is being lowered. Table 3 reports the three gate decisions.

[Figure pending — drafted Week 3. Plots $\log_{10}(\textit{params})$ on x-axis vs val_loss on y-axis, with the three Mini/Base/Medium points and the calibrated gate threshold per scale shown as a horizontal step function. The visual signal is the widening gap between the trained val_loss curve and the gate-step function.]

Figure 1: IntLLM scaling-law curve (Mini/Base/Medium val_loss vs parameter count, with calibrated gate threshold overlay). Source: `paper/intllm/results/training_intllm-{mini,base,medium}.json`.

The margins widen 0.118 → 0.210 → 0.279 nat across the chain. Equivalently, in held-out perplexity: PPL = 80.0 → 54.1 → 41.3, a 1.94× reduction end-to-end. Figure 1 (deferred to Week 3 calendar) plots the three points on the standard log-parameter axis.

We treat the monotonic-widening signal as the principal scaling-chain claim because absolute val_loss is sensitive to tokenizer choice and validation-stream sampling decisions that other ternary papers do not hold fixed; the relative widening, in contrast, is invariant to those choices as long as the same evaluation protocol is applied to all three scales (which it is, by construction of Section 4.1).

5.2 Wikitext and zero-shot canonical benchmarks

Beyond the training-time val_loss gates, we evaluate each checkpoint on the standard `lm-evaluation-harness v0.4.11` 8-task zero-shot suite plus Wikitext perplexity. All measurements use the harness's published task definitions with no limit, no custom protocol, and the harness SHA pinned in `eval/HARNESS_SHA` per the research-integrity rule R1 of Section 4.5. Table 4 reports the results.

The pure language-modeling axis (Wikitext word-PPL, Wikitext bits-per-byte, lambada PPL, lambada accuracy) shows clean monotonic scaling end-to-end:

- Wikitext word-PPL drops 343.0 → 201.1 → 138.4 (−60% end-to-end, 1.41× per scale step).
- Wikitext bits-per-byte drops 1.575 → 1.431 → 1.330 (−16% end-to-end), the metric that is invariant under tokenizer choice and therefore directly comparable to other tokenizer families [Biderman et al., 2023].
- Lambada PPL drops by an order of magnitude per scale step (51,121 → 16,729 → 5,277).
- Lambada accuracy scales 16× from Mini to Medium (0.001 → 0.023), albeit from a near-zero baseline.

The multi-choice reasoning columns (arc-easy, arc-challenge, hellaswag-norm, piqa, openbookqa, plus winogrande not shown for space) mostly hover within ±1-2 standard errors of their random-chance baseline at this parameter range. This is consistent with the Chinchilla [Hoffmann et al., 2022] and Pythia [Biderman et al., 2023] observations that sub-100M-parameter models cannot meaningfully exceed random on these tasks; modeling capacity must reach ∼1B before structured-reasoning skills emerge. We report the numbers for completeness and reproducibility, not as a competitive benchmark claim.

5.3 Interpretation: what the scaling chain does and does not establish

The Phase D scaling chain establishes three things:

1. **1.58-bit ternary training-from-scratch is loss-monotone-stable at the 22M-74M parameter range.** Gate margins widen monotonically across three independent training runs at three distinct configurations. There is no scale at which adding parameters and training tokens fails to reduce val_loss; the trend is robust to the tokens-per-parameter spread (22.86 to 24.40) within Chinchilla tolerance.
2. **The literature scaling-law assumption that Chinchilla-proportional token budgets are compute-optimal extends from FP16 [Hoffmann et al., 2022] to ternary at this scale.** BitNet b1.58 [Ma et al., 2024] validated scaling laws at 700M+; we close the gap downward to 22M-74M, the parameter range most relevant for embedded and kernel-context deployment.
3. **Pure language-modeling perplexity reductions are ∼60% end-to-end across this 3.5× parameter range,** indicating that the 1.58-bit training recipe captures real signal rather than being a trivial fit. Bits-per-byte (the tokenizer-invariant metric) drops 1.575 → 1.330, a regime where each 0.1 nat improvement is materially difficult.

The chain does *not* establish frontier performance on multi-choice reasoning benchmarks; the model is too small. It also does not establish that ternary training matches FP16 training at the same parameter count (it doesn't; FP16 baselines

Table 4: IntLLM canonical benchmark results across the scaling chain. Pure-language-modeling tasks (Wikitext, lambada) show clean monotonic improvement; multi-choice reasoning tasks (the rightmost six columns) hover near random-baseline levels at this parameter range, consistent with the literature observation that sub-100M models do not meaningfully exceed random on these tasks [Biderman et al., 2023, Hoffmann et al., 2022]. The contribution of the scaling chain is the trend in the first three columns, not absolute wins on the multi-choice columns.

	Wikitext		lambada		Zero-shot accuracy (↑)				
Config	word-PPL ↓	bpb ↓	PPL ↓	acc ↑	arc-e	arc-c	hella-norm	piqa	openbookqa
Mini	343.0	1.575	51,121	0.001	0.306	0.181	0.255	0.559	0.110
Base	201.1	1.431	16,729	0.007	0.319	0.192	0.262	0.555	0.128
Medium	138.4	1.330	5,277	0.023	0.341	0.189	0.258	0.571	0.130

such as Pythia-160M [Biderman et al., 2023] and SmolLM2-135M [Allal et al., 2024] achieve materially better Wikitext word-PPL at comparable parameter counts). The contribution is the *combination* of (a) ternary precision, (b) smaller-than-existing- small-LM scale, (c) kernel-deployable binary footprint (Section 8), and (d) the bilingual extension (Section 6) — see the synthesis table in `docs/INTLLM_RELATED_WORK.md` §7.1 for the multi-axis positioning.

6 Bilingual Extension and Q5 Baseline (C2)

The IntLLM scaling chain of Section 5 is English-only: the SlimPajama-6B training stream contains negligible non-English content, and all evaluation tasks (Wikitext, lambada, the zero-shot suite) are English. Contribution C2 extends IntLLM to Indonesian-plus-English bilingual training via a new Phase E1 corpus and a Q5 baseline measurement. We do *not* claim a fully-trained bilingual production model in this paper; that is deferred to Phase E3 (Section 10). The C2 claim is narrower: *a clean, license-attributed, reproducible bilingual corpus exists, and a Mini-scale baseline measurement against it establishes the floor that all subsequent E2 ablations (Section 7) compare against.*

6.1 Phase E1 bilingual corpus v1.0

The corpus combines two Indonesian sources and one English source:

- **Indonesian Wikipedia** (`wikimedia/wikipedia` ID configuration via HuggingFace): 662,929 documents kept after exact-hash deduplication (0.40% duplicate rate), 1.06 GB raw text.

Table 5: Phase E1 bilingual corpus v1.0 composition. ID side is materialized on disk under `data/phase_e/corpus_id_dedup_exact/`; EN side is streamed from HuggingFace at training time (no on-disk pre-stage). Numbers reproducible via the manifest files committed alongside each ID-side parquet shard. Deduplication uses exact SHA-256 hashing on NFKC-lowercase-whitespace-normalized text.

Source	Docs kept	Size	Tokens
ID Wikipedia	0.66 M	1.06 GB	∼0.42 B
ID FineWeb-2	10.00 M	37.86 GB	∼14.98 B
ID total	**10.66 M**	**38.92 GB**	**∼15.40 B**
EN SlimPajama-627B	(streamed)	(streamed)	∼10.27 B
Combined	**∼10.66 M+EN**	**38.92 GB**	**∼25.67 B**

- **Indonesian FineWeb-2** (`HuggingFaceFW/fineweb-2` `ind_Latn` subset): 9,999,982 documents kept (∼0% duplicate rate; the upstream is already deduplicated at scale), 37.86 GB raw text.
- **English SlimPajama-627B-Reupload** (`gmongaras/SlimPajama-627B_Reupload`): streamed at training time rather than pre-staged on disk, capped at the bilingual budget (Table 5).

The combined Indonesian side after exact-hash deduplication is ≈10.66 M documents at 38.92 GB, tokenizing to ≈15.40 B tokens under the Mistral v3 BPE (Section 3.4). At the design 60:40 ratio, the English side caps at ≈10.27 B tokens for a combined total of ≈25.67 B tokens. Table 5 summarizes the composition; Figure 2 (deferred to Week 3) plots it as a pie chart for the camera-ready version.

A mechanical consistency gate `make verify-bilingual-corpus` checks 8 invariants (file presence, manifest internal consistency, dedup-rate bounds, total-token range, ID/EN ratio bounds,

[Figure pending — drafted Week 3. Pie chart of corpus composition: ID FineWeb-2 (58.4%), EN SlimPajama (40.0%), ID Wikipedia (1.6%). Source: per-shard `_manifest.json` + `python/phase_e/intllm_en.py` EN-cap arithmetic.]

Figure 2: Phase E1 corpus v1.0 composition by tokens. Source: `data/phase_e/corpus_id_dedup_exact/*/_manifest.json`.

license attribution presence, schema versions, sha256 hash table integrity) in < 1 second; a pre-commit hook fires this gate conditionally when staged changes touch any corpus manifest or the `intllm_en.py` configuration shim. License attribution (CC BY-SA 4.0 for Wikipedia, ODC-By 1.0 for FineWeb-2, Apache 2.0 for SlimPajama-627B) is preserved in `NOTICE_BILINGUAL_CORPUS_V1` at the repository root for downstream model release.

The corpus is **0%-synthetic by design**: every token in the ID side comes from human-authored sources, and the EN side reuses an already-published large-scale public web corpus. We deliberately exclude synthetic generation (Gemma 4 / Lapis 2.5 outputs) from this v1.0 corpus to maximize licensing clarity for downstream model release; synthetic augmentation is a Phase E1.5 deferred deliverable not exercised in this paper.

6.2 Q5 Mini-scale baseline measurement

Q5 is a single Mini-scale training run on the bilingual corpus, measured per Section 4.2's protocol: 24,000 steps × 8 batch × 1024 sequence = ≈196 M tokens at 60:40 ID:EN, on a single RTX 4090 Laptop GPU. The run takes 42.0 minutes wall-clock. After training, validation losses are measured separately on Indonesian and English held-out streams (50 batches each at distinct seeds) to compute the cross-lingual coherence ratio $\rho = \max(\ell_{\text{ID}}, \ell_{\text{EN}}) / \min(\ell_{\text{ID}}, \ell_{\text{EN}})$.

Table 6 reports the result.

The Q5 baseline establishes the bilingual floor for this paper: every E2 ablation in Section 7 is gate-evaluated against `val_loss(EN, seed=999, 50 batches)` (Q5 reuses the EN seed=998 stream; the seed alignment is documented in Section 4.4, with the 50-batch `seed=999` EN protocol used by ablations being a deliberately distinct sample).

Table 6: Q5 bilingual baseline at Mini scale (21.9 M params, 24,000 steps on Phase E1 corpus v1.0 at 60:40 ID:EN). val_loss measured on 50-batch held-out streams: `seed=999` for ID, `seed=998` for EN. Source: `paper/intllm/ablations/q5_bilingual_baseline.json`.

Metric	Value
`val_loss(ID)`	2.679
`val_loss(EN)`	4.732
$\rho = \max / \min$	**1.77**×
`PPL(ID)`	14.6
`PPL(EN)`	113.5
training wall-clock	42.0 min

6.3 Honest interpretation: why ID fits 7.8× better than EN

The numerical surprise of Q5 is that `val_loss(ID)` is materially lower than `val_loss(EN)` despite the 60:40 mix weighting Indonesian only 1.5× over English. In perplexity terms the gap is 7.8× (14.6 vs. 113.5). We attribute this to a combination of three effects, none of which constitute "bilingual interference" in the sense the multilingual quantization literature uses the phrase [Anonymous, 2026]:

1. **Indonesian Wikipedia + FineWeb-2 has higher template redundancy than SlimPajama EN.** Indonesian Wikipedia in particular has a long tail of stub pages with near-identical structural prefixes ("Kelahiran", "Kematian", "Pranala luar", etc.). The exact-hash deduplication of Section 6.1 catches *exact* duplicates (0.40% on Wikipedia) but leaves *near*-duplicates that share many tokens of structural boilerplate; the model picks up on this and predicts those tokens with very low cross-entropy.

2. **Mistral v3 BPE is English-trained.** Indonesian text is within the BPE's Latin-script coverage but typically requires more sub-token pieces per word than equivalent English content. This inflates the per-token information content of English (each token carries more uncertainty) while compressing it for Indonesian, biasing val_loss in Indonesian's favor under same-tokenizer comparison.

3. **The English side at 80 M tokens (40% of 196 M) is below Phase D Mini's saturation point.** Phase D Mini v2 reached val_loss 4.382 at 491.5 M EN tokens (Section 5). Q5's English side trains on 80 M tokens, ≈16% of that budget; the

resulting val_loss 4.732 is consistent with a half-trained English-only Mini, *not* an architecturally degraded multilingual model.

The implication for Phase E2 ablation interpretation is important: a coherence ratio of $\rho = 1.77\times$ at the no-feature baseline is a *property of the corpus and tokenizer choice*, not evidence that bilingual ternary training fundamentally fails. The language-conditioned-design literature [Anonymous, 2026] would predict $\rho \to 1.0$ as a target for an ideal multilingual model; in our setting, $\rho \to \sim 1.5$ may be the practical floor under the SlimPajama-EN-vs-Wikipedia/FineWeb-2-ID corpus asymmetry, and the gap between 1.77 and 1.5 should be interpreted as the headroom that language-conditioned design (Phase F.9) is responsible for closing.

To IntLLM's contribution stack specifically, Q5 is the **first fully-public bilingual ternary LM training run at the 22 M-parameter scale**. The corpus, training script, and result JSON are all artifact-committed and reproducible; the next research team can rerun Q5 in ∼42 minutes on commodity laptop hardware and verify the $\rho = 1.77$ baseline before attempting their own bilingual extensions.

7 Honest Negative Results on Outlier-Handling Features (C4)

This section reports two outlier-handling features that we ported from the post-hoc-quantization literature and ablated against the Q5 bilingual baseline of Section 6.2. **Both features FAIL their adoption gates**: balanced_calib produces a quantization error 83× worse than the upstream baseline on outlier channels, and Hadamard rotation regresses val_loss by +0.12 nat against a gate that demanded a $\geq +0.05$ nat improvement. We report the failures honestly per the research-integrity rule R6 of Section 4.5, with a three-cause attribution per feature (Sections 7.2 and 7.3) and a Phase F future-work pointer (Section 7.5). The methodology of this section is itself a contribution: small-scale training-from-scratch ternary literature has not, to our knowledge, published systematic negative-result attributions of this kind, and the resulting fence-posts inform what does and does not transfer from post-hoc large-scale quantization to small-scale training-from-scratch.

7.1 What we ablated and why

Two literature-grounded outlier-handling features were selected as high-prior-probability adoption candidates per the original Phase E plan v1.1, before any ablation evidence was collected:

1. **balanced_calib** (E2.4 in the Phase E plan): per-channel calibrated activation-quantization scales derived from a `BitLinearStatTracker` accumulator that captures the running-max activation magnitude per channel during training, plus a top-K% adaptive bit allocation that promotes the most outlier-prone 5% of input channels from 8-bit to 10-bit. Literature precedent: SmoothQuant [Xiao et al., 2022], AWQ [Lin et al., 2023], GPTQ [Frantar et al., 2022].

2. **Hadamard rotation** (E2.1 in the Phase E plan): a fixed Walsh-Hadamard orthogonal rotation applied as a forward pre-hook on the inputs of every `block.attn.o_proj` BitLinear. Spreads outlier energy uniformly across the hidden dimension before quantization, so per-token absmax quantization grids fit the post-rotation distribution more tightly. Literature precedent: QuaRot [Ashkboos et al., 2024], SpinQuant [Liu et al., 2024]. We restrict insertion to `o_proj` per the Phase E E2.0 closure question Q1, since HGRN's `i_proj`/`f_proj`/`g_proj` are gated-linear-recurrence projections structurally distinct from QKV; rotating them is non-canonical under the published QuaRot recipe.

Both features are gate-evaluated under the Q5 baseline protocol of Section 4.2 (24,000 steps × 8 batch × 1024 sequence on the bilingual stream at 60:40 ID:EN). The adoption gates are mechanical and were committed before the experiments ran:

- **balanced_calib gate (Phase E2.4.C, Option C):** `outlier_global_reduction` ≥ 0.10 on a 1,000-batch bilingual eval set, where the metric measures the relative MSE reduction between the upstream baseline activation quantizer and $q_{\text{calibrated}}$, restricted to channels promoted to 10-bit. The rationale and full schema are committed in `FJQ_PHASE_E_E2_4_C_METRIC_SPEC.md` v1.0.

- **Hadamard gate (Phase E2.1.4):** `val_loss(EN, seed=999, 50 batches)` $<$ Q5 baseline $- 0.05$ nat $= 4.732 - 0.05 = 4.682$.

Table 7 reports the gate decisions; the following two subsections give the full 3-cause attribution per feature.

Table 7: Mini-scale outlier-handling ablation summary. *Q5 baseline* is the no-feature row from Section 6.2. Δ *vs Q5* for the val_loss columns is positive when the ablation IMPROVES on the baseline (negative = regression). The `outlier_global_reduction` column is only defined for balanced_calib (the metric requires saved calibration maps). All training runs use identical step budget and bilingual stream; training trajectories agree with Q5 to within seed-variance, so the val_loss differences are attributable to the feature itself.

	val_loss(EN)	Δ vs Q5	outlier_red. gate
Q5 baseline	4.732	—	
balanced_calib (E2.4)	4.834	−0.10	−82.13 **FAIL**
Hadamard (E2.1)	4.852	−0.12	

7.2 E2.4 balanced_calib: 83× worse than baseline on outlier channels

The balanced_calib metric outcome is unambiguous: `outlier_global_reduction` = −82.13 against a gate of $\geq +0.10$. The negative sign means the calibrated quantizer is *worse* than the baseline on the very channels it was designed to help: the calibrated mean MSE on outlier channels is 83.13× the baseline mean MSE. `global_mean_reduction` (across all channels, not just outliers) is −148.27, indicating the calibrated quantizer is also ∼149× worse on bulk channels. Per-layer breakdown shows the failure is uniform across all 37 BitLinear sites, not driven by a few pathological layers.

The three-cause attribution committed in `FJQ_PHASE_E_E2_BILINGUAL_CALIB_DECISION.md` v1.0 §3 reads:

1. **All-time-max `running_max` accumulator captures non-stationary peaks.** The accumulator is `torch.maximum` over every forward call across all 24K training steps. Early training has random-init activation peaks 10-100× larger than steady-state. The accumulator captures these and never decays, so the calibrated scale is locked to a worst case the late-training model rarely produces. Per-layer evidence: `attn.o_proj running_max` ranges 11–20 across the 6 layers, while typical late-training $|x|$ on the same channel is ∼0.1-0.5 post-RMSNorm. The calibrated scale = 127/running_max ≈ 6-12 is far too coarse for the steady-state distribution; the baseline per-token scale = 127/0.5 ≈ 254 is much finer.

2. **RMSNorm-bounded BitLinear inputs reward per-token adaptation, not per-channel calibration.** Upstream BitLinear runs RMSNorm before `activation_quant`, bounding inputs adaptively per-token. The baseline per-token `activation_quant` benefits: `scale_t = 127 / max(|x_norm[t,:]|)` is large for the typical small-magnitude rows that emerge late in training, granting fine-grained quantization. `q_calibrated` cannot adapt — it locks to the calibration constant.

3. **Outlier-bit allocation lever is ineffective when scale is fundamentally too coarse.** The 5% top-K → 10-bit promotion gives outlier channels MORE bits, but only addresses GRANULARITY error within the channel's chosen scale; it does not fix the locked-coarse-scale problem. AWQ [Lin et al., 2023] and GPTQ [Frantar et al., 2022] both note this dependency; our negative result is consistent with this literature observation.

7.3 E2.1 Hadamard rotation: +0.12 nat regression on val_loss

The Hadamard rotation outcome is also unambiguous: `val_loss(EN)` = 4.852 against the Q5 baseline 4.732, a regression of +0.120 nat against a gate demanding a $\geq +0.05$ nat improvement (FAIL by +0.17 nat past gate, in the wrong direction). Held-out training trajectories match Q5 within ±0.02 nat at every checkpointed step, confirming the failure is attributable to the rotation itself rather than to training-trajectory disturbance from the bilingual sampler or the Hadamard pre-hook.

The three-cause attribution committed in `FJQ_PHASE_E_E2_HADAMARD_DECISION.md` v1.0 §3 reads:

1. **Training-from-scratch loses Hadamard's primary benefit.** QuaRot [Ashkboos et al., 2024] and SpinQuant [Liu et al., 2024] apply rotation *post-hoc* to a pre-trained model that is then re-quantized. The pre-trained model already has stable, learned representations; rotation reduces outlier-driven quantization error WITHOUT requiring weight adaptation. In our setup we train from scratch with the rotation in place; the training-from-scratch math equivalence $y = \text{quant}(W) \cdot \text{quant}(Hx) \approx W \cdot Hx = (WH)x$ implies the model learns $W_{\text{eff}} = WH$ rather than W, and spends gradient updates fighting the rotation rather than benefiting from outlier reduction. The Mini-scale 491M-token-equivalent compute budget is far less than the budgets where weight-learning saturates against the rotation handicap.

2. **HGRN attention output may not have transformer-style outlier concentration.** The QuaRot/SpinQuant outlier story is about transformer self-attention's QKV projection inputs (post-RMSNorm hidden states), where a few channels concentrate magnitude due to the softmax$(QK^T/\sqrt{d})$ attention pattern emphasizing certain feature dimensions. HGRN's gated linear recurrence is structurally different: gate values $g_t = \sigma(g_{\text{proj}}(x_t))$ and forget values $f_t = \sigma(f_{\text{proj}}(x_t))$ modulate the recurrence smoothly, without softmax-induced sharpness. The activation arriving at `o_proj` is the post-recurrence accumulator state, not a softmax-attention-weighted sum. We did not empirically measure outlier concentration on `o_proj` inputs before running this ablation; in hindsight that pre-flight measurement should have been a hard prerequisite (Phase F.6.1 future-work).

3. **Q1 closure's "rotate `o_proj` only" may have been overly conservative.** The full QuaRot recipe rotates inputs to ALL projections including QKV-equivalent paths; HGRN's `i_proj`/`f_proj`/`g_proj` inputs are also post-RMSNorm hidden states, just like QKV inputs in transformers. The same outlier pattern *could* be present there. Our ablation tested only 1/4 of the candidate rotation sites in HGRN attention; a future ablation that rotates all four sites might recover the literature's positive result.

Figure 3 (deferred to Week 3) plots a synthetic single-channel-spike input through the same `HadamardRotation` module and shows it DOES correctly spread outlier energy mathematically: the max-to-RMS ratio drops from $\sqrt{d_{\text{model}}} = 8$ to $\sim$1 after rotation. The mathematical mechanism works as designed; the training-from-scratch + Mini-scale + bilingual setting is what fails to translate this mechanism into val_loss improvement.

[Figure pending — drafted Week 3. Bar chart of per-channel $|x|$ before rotation (one tall bar at channel 0, rest near-zero) and after rotation (uniform spread at $1/\sqrt{d}\cdot$max). Demonstrates the Hadamard rotation works mathematically; the negative training- from-scratch result is NOT due to broken math. Source: `python/phase_d/tests/test_quant.py::test_hadamard_outlier_suppression_on_concentrated_input` *unit-test fixture extended to dump pre/post arrays.]*

Figure 3: Walsh-Hadamard rotation correctly spreads single-channel outlier energy on synthetic input (max/RMS ratio drops 8 $\rightarrow$ 1 at $d = 64$). The mechanism works; training-from-scratch at Mini scale is what fails to capitalize on it. Source: `HadamardRotation` unit-test fixture.

7.4 Common pattern across both failures

Both features come from the *post-hoc quantization* literature: QuaRot, SpinQuant, SmoothQuant, GPTQ, and AWQ all apply their respective modifications to a pre-trained model that is subsequently re-quantized. None of these methods has, to our knowledge, been ablated under the conditions of this paper: training-from-scratch + Mini-scale (22 M params) + bilingual data. Our two negative results suggest a meta-pattern:

Table 8: Three-cause attribution summary. Cause 1 of each feature identifies the same fundamental mismatch: literature precedents are post-hoc, our setting is training-from-scratch. Causes 2 and 3 are feature-specific.

#	balanced_calib (E2.4)	Hadamard (E2.1)
1	all-time-max captures non-stat. peaks	training-from-scratch los
2	RMSNorm reward per-token adapt. scale	HGRN may lack QKV-s
3	outlier-bit lever needs correct scale	rotate-`o_proj`-only may

> *Outlier-handling features designed for post-hoc quantization of pre-trained large transformers do not transfer cleanly to training-from-scratch ternary at small scale.*

The meta-pattern is a hypothesis, not a theorem; we test only two features. Confirming it would require additional ablations (Section 7.5). What our two failures *do* establish is that the literature's positive results in their post-hoc large-scale regime do not provide *prima facie* evidence for adoption in our regime, and that mechanically-gated ablations on Q5-comparable budget reveal the gap quickly.

Table 8 summarizes the three-cause attribution columns side-by-side; the structural commonality is the "post-hoc literature precedent vs. training-from-scratch setting" mismatch in cause 1 of each.

7.5 Phase F future-work pointers

The negative results inform four committed Phase F future-work entries (`FJQ_PHASE_F_TAX_VERTICAL_ROADMAP.md` v1.3 §4.1):

- **F.5 — post-training PTQ-style calibration on held-out data.** Replace the all-time-max accumulator with a SmoothQuant-style [Xiao et al., 2022] held-out post-training calibration on a fixed batch from the trained Mini checkpoint. Tests whether the balanced_calib failure was about WHEN calibration happens (during-training accumulator) rather than WHAT calibration is computed (per-channel scale).

- **F.6 — post-hoc QuaRot on the Q5-trained Mini checkpoint.** Apply the canonical QuaRot recipe (fuse Hadamard into adjacent weights via algebraic identity, re-evaluate without re-training) to the Q5-trained checkpoint. Tests whether the Hadamard failure was about TRAINING-from-scratch vs. POST-hoc, controlling for the architecture (HGRN-Bit unchanged).

- **F.6.1 — outlier-concentration measurement on Q5-trained checkpoint o_proj inputs.** A pre-flight measurement of the per- channel max-to-RMS ratio histogram on captured `o_proj` inputs from the trained Mini checkpoint. If the ratio is uniformly ∼1 (no outlier concentration), F.6 post-hoc rotation has nothing to spread either; if there is meaningful concentration, F.6 likely recovers a positive result. This pre-flight is the hindsight prerequisite Section 7.3 cause 2 should have satisfied before E2.1 was run.

- **F.7–F.9 — additional outlier-handling features cut from this paper's scope.** FP8 mixed-precision (F.7), FP16 distillation (F.8), and language-conditioned design (F.9) were originally planned as Phase E2 ablations and cut to Phase F under the Path A scope decision (`FJQ_PHASE_E_PATH_A_PAPER_OUTLINE.md` v1.0). The cut rationale is that with two negative results in a row from features structurally similar to those three (all post-hoc-literature- inspired), the marginal evidence per additional ablation is low; cut the losses, ship the negative-results paper, pursue the alternatives in Phase F where larger-scale and post-hoc protocols are available.

We treat this as a successful Phase E2 outcome rather than a failure of the methodology: the two negative results have prevented an inflated paper claim, the three-cause attributions give the next research team specific hypotheses to test, and the four Phase F entries constitute a more well-posed research agenda than the original "ablate five features and combine the wins" plan that the empirical evidence has refuted.

8 In-Kernel Deployment via FajarOS Nova (C3)

The architecture (Section 3) and training methodology (Section 4) of IntLLM are designed so the trained checkpoint can run *inside* the operating-system kernel address space — no userspace process, no syscall boundary during inference, no heap allocation. This section reports the deployment of the Phase D Medium checkpoint as a kernel-resident inference path inside FajarOS Nova v3.9.0 (release dated 2026-04-24, three-day-soak validated), including the 4-invariant regression gate that we run on every commit to verify the path's mechanical stability.

8.1 Kernel-path architecture

The IntLLM kernel-path is implemented in three Fajar Lang modules, all marked `@kernel` and therefore subject to the compile-time safety constraints of Section 9: no heap allocation (`KE001`), no Tensor type construction (`KE002`), and no calls to `@device`-marked functions (`KE003`). The three modules form a layered pipeline:

1. `matmulfree.fj` (kernel ops module). Implements the integer-only ternary BitLinear inner loop `km_mf_bitlinear_packed`, the 257-entry sigmoid and SiLU LUTs, and the i64 fixed-point RMSNorm. All operations are integer arithmetic; no FP hot path, no heap allocation, deterministic execution.

2. `fjm_v9.fj` (model parser). Parses the v9 ternary checkpoint format (`.fjm v9`) produced by `intllm.export.export_fjm_v9` during Phase D training-export. Includes the IEEE-754 f32 to i64×1000 fixed-point decoder for the small number of FP-stored constants (RMSNorm β scales, LUT endpoints) that the v9 format retains in non-ternary precision.

3. `tfm_matmulfree.fj` (forward orchestrator). Wires the `matmulfree` kernel ops into a transformer-shaped decode loop (`tfm_mf_generate`); embedding lookup, RoPE apply, attention block, FFN block, RMSNorm, output sampling. Invoked from the FajarOS Nova shell via `cmd_ask`, which dispatches Phase D models (v9 magic in the loaded checkpoint header) through this path while routing legacy v8 RWKV models through the older `tfm_generate` path.

The three modules together total ∼1,500 lines of Fajar Lang code; kernel-side, with no userspace dependencies, runtime image budget (Section 8.3), and full Fajar Lang `@kernel`-context type checking (Section 9).

The full inference flow from user input to output token sequence is:

```
shell prompt: nova>
  user types: model-load nvme 0
    -> mdl_parse_header(disk_buf)
       -> dispatches v9 (HGRNBit)
       -> fjm_v9.fj reads weights into
          @kernel-static buffer
  user types: ask hello
    -> shell cmd_ask routes to
       tfm_mf_generate(prompt_tokens)
       -> embedding -> 6 HGRN blocks ->
          lm_head -> sample
       -> emits tokens to serial console
```

This inference flow runs entirely in kernel-context: at no point does control transfer to userspace, no syscalls execute, no heap allocation occurs (the model weight buffer is statically allocated at boot per the kernel-context constraint).

8.2 4-invariant regression gate

A Makefile target `make test-intllm-kernel-path` exercises the inference flow inside QEMU on every commit, gating mechanical stability of the path as an artifact of the build. The gate uses QEMU with the FajarOS Nova ISO booted from CD-ROM (`-boot order=d`) and an attached NVMe disk image carrying a tiny synthetic v9 model checkpoint, waits for the shell prompt, sends `model-load nvme 0` and `ask hello` commands via the serial console, captures the serial-console output to a log file, and checks four invariants:

1. **No fault markers.** The serial log contains neither `EXC:` (CPU exception) nor `PANIC:` (kernel panic) markers; the inference path completes without triggering the kernel's mechanical-stability fail-safes.

2. **v9 header parsed.** The log contains `[OK] Model header loaded`, confirming `mdl_parse_header` accepted the v9 magic and dispatched the checkpoint to the `fjm_v9.fj` parser branch.

3. `cmd_ask` **dispatched to** `tfm_mf_generate`**.** A serial-log marker emitted from inside `cmd_ask`'s dispatch logic confirms the v9 routing decision took the IntLLM kernel-path rather than the legacy v8 RWKV path.

Table 9: FajarOS Nova v3.9.0 binary footprint. The *kernel ELF* is the unstripped output of the Fajar Lang LLVM backend (`build/fajaros-llvm.elf`); the *ISO* adds GRUB and boot-stage loaders for QEMU/UEFI booting. The *model* column is the ternary checkpoint loaded at runtime via NVMe (not part of the kernel binary; loaded into a static @kernel buffer at `model-load` time). Total budget is kernel + model ≤ 16 MB per the kernel-context constraint of Section 3.

Component	Size
FajarOS Nova v3.9.0 kernel ELF (unstripped)	1.66 MB
of which: `matmulfree.fj` kernel ops	∼0.18 MB
of which: `fjm_v9.fj` parser	∼0.04 MB
of which: `tfm_matmulfree.fj` forward	∼0.06 MB
of which: rest of FajarOS Nova kernel	∼1.38 MB
ISO image (with GRUB + boot loaders)	13.82 MB
Phase D Medium ternary model (`.fjm v9`)	∼14.7 MB*

* Estimated as 74.5 M params ×1.58 bits/8 = 14.71 MB ternary-packed weights, plus ∼0.4 MB of full-precision β scales and embedding/lm_head matrices retained at higher precision in the v9 export format. Direct measurement of an exported Medium model file is deferred to the camera-ready version (Path A outline §5 "MUST-HAVE before submission" checklist).

4. **Shell recovered after** `ask`**.** The log contains a return to the `nova>` prompt after the inference completes, confirming the kernel state was not corrupted by the inference run and the shell remains responsive.

The gate explicitly does NOT verify token-level coherence quality of the generated output: the synthetic test model used by the regression gate is a tiny path-verification fixture, not a quality-evaluation model. Generation-quality verification is the job of the offline Phase D Mini/Base/Medium training results (Section 5); the kernel-path gate verifies *the path itself works*, not *the model is good*.

8.3 Binary footprint and budget

Table 9 reports the static binary footprint of the FajarOS Nova v3.9.0 build artifact set, against the ≤ 16 MB total-budget constraint of contribution C3.

The kernel ELF at 1.66 MB plus the Phase D Medium ternary model at ∼14.7 MB sums to ∼16.36 MB, slightly exceeding the ≤ 16 MB design budget at the Medium scale. Two of three deployment paths satisfy the budget: (a) the Mini-scale model (∼4.3 MB ternary) plus kernel ELF totals ∼6.0 MB, well under budget; (b) the Base-

[Figure pending — drafted camera-ready. Stacked bar chart of the 16 MB budget: kernel ELF (1.66 MB), matmulfree.fj kernel ops (∼0.18 MB), fjm_v9.fj parser (∼0.04 MB), tfm_matmulfree.fj forward (∼0.06 MB), Mini/Base/Medium ternary weight totals (∼4.3 / ∼9.2 / ∼14.7 MB), free headroom. Source: `ls -l build/fajaros-llvm.elf` + Fajar Lang object-file introspection per kernel module + ternary-pack arithmetic per `intllm.export.export_fjm_v9`.]

Figure 4: FajarOS Nova v3.9.0 binary budget breakdown by component. Source: `build/` artifact inspection; per-module sizes from ELF section table.

Table 10: IntLLM kernel-path runtime metrics. Measurement scaffolding (`make test-intllm-kernel-path-runtime`) is shipped as part of the v3.9.0 release; the four cells below populate from the `build/test-intllm-kernel-path.log` per QEMU run, with watts captured by an external USB-C power monitor on bare-metal Radxa Q6A when that platform's deferred-from-V31 partner integration completes. **PEND** = pending measurement at camera-ready.

Config	tok/s (QEMU)	power (W, bare metal)
Mini (22 M)	PEND	PEND
Base (46 M)	PEND	PEND
Medium (74 M)	PEND	PEND

scale model (∼9.2 MB ternary) plus kernel ELF totals ∼10.9 MB. Medium-scale deployment requires either kernel-side stripping (currently disabled for debugging) or v9-format compression (not yet implemented); we report the unmodified Medium total honestly and treat the marginal overrun as a calibration target for v9-format follow-up work.

Figure 4 (deferred to camera-ready) plots the budget breakdown stacked by component.

8.4 Runtime metrics: tok/s and watts

Table 10 reports the runtime metrics for the IntLLM kernel-path on FajarOS Nova v3.9.0. Static binary measurements in Section 8.3 are direct artifact inspections; the runtime measurements below require fresh capture on the target hardware (RTX 4090 Laptop CPU host running QEMU; bare metal capture deferred to a Phase F deployment-platform partner that the V31 cycle does not have access to).

The 4-invariant kernel-path gate of Section 8.2 is sufficient for the *path-verification* claim of contribution C3 — the inference runs end-to-end inside @kernel context without violating the compile-time safety invariants of Section 9. Runtime performance numbers are an additional axis on which we make no camera-ready claim until measurement is complete; the paper is explicit that C3's contribution is the *architectural integration*, not a benchmark beat against userspace inference engines like `llama.cpp` or `Bitnet.cpp` [Xu et al., 2025] that operate in fundamentally different deployment regimes.

8.5 Honest scope of contribution C3

What the kernel-path establishes:

1. Phase D ternary checkpoints can be loaded and inferenced *without leaving the operating-system kernel address space* on a real (FajarOS Nova) operating system.
2. The four mechanical-stability invariants pass on every CI run, gating regressions to the inference path on every commit.
3. The static binary footprint at the Mini and Base scales fits the $\leq$ 16 MB design budget; Medium overruns marginally and identifies v9-format compression as a follow-up calibration target.

What the kernel-path does NOT establish:

1. Token-level coherence-quality verification — that is the job of Section 5 and Section 6.2, not the kernel-path gate.
2. Bare-metal performance numbers (tok/s, watts) — pending measurement; deferred to a future deployment-platform partner.
3. Production-grade hardening of the FajarOS Nova kernel itself — the kernel passes its own 35-test regression suite (per the FajarOS Nova v3.9.0 release notes) but is research-grade, not formally verified in the seL4 [Klein et al., 2009] sense.

This honest delimitation of C3 is consistent with the research-integrity rules R3 and R6 of Section 4.5: we report what we have measured mechanically, leave gaps explicit, and avoid implying claims for which we lack the measurement infrastructure.

9 Compiler-Verified Safety (C5)

The Fajar Lang compiler enforces a four-context type system that rules out kernel-context safety violations at compile time. The `@kernel`, `@device`, `@safe`,

and `@unsafe` annotations partition functions and call sites into deployment domains; the analyzer pass between parser and code generator validates that no function-call edge crosses a domain boundary in a way that would violate the target deployment's invariants. This is the contribution that distinguishes IntLLM from "ternary in userspace" deployments [Ma et al., 2024, Xu et al., 2025]: the kernel-context invariants of Section 8 are mechanically enforced by the compiler rather than relied on as a documentation-grade convention.

9.1 The four-context type system

Every function in a Fajar Lang program carries exactly one of four context annotations, with the following operational meaning:

- `@safe` (default if no annotation specified): unrestricted within the safe-by-default subset; no hardware access, no raw pointers, no kernel privileged operations, no direct tensor construction. Userspace business-logic code lives here.
- `@kernel`: kernel-context. Allowed: stack allocation, raw pointer dereference, page-map and IRQ-register intrinsics (`alloc!()`, `irq_register!()`, etc.), syscalls *into* the kernel from `@kernel` functions. Forbidden: heap allocation (`String`, `Vec`), tensor construction (`zeros`, `ones`, `relu`, etc.), calls to `@device`-marked functions.
- `@device`: device / accelerator context (GPU, NPU, embedded inference accelerator). Allowed: tensor operations and device-side intrinsics. Forbidden: raw pointer dereference outside explicit `@unsafe` blocks, hardware-register intrinsics (`irq_register!`, `port_write!`, `map_page!`), calls to `@kernel`-marked functions.
- `@unsafe`: full access; bypasses all of the above restrictions. Used only at deliberate intent points (FFI shims, specialized kernel modules) and clearly delimited.

Table 11 reproduces the operation-by-context matrix from the Fajar Lang specification (CLAUDE.md §5.3) showing which operations are allowed in each context. The type-system-enforcement of this matrix is what gives the kernel-LLM deployment of Section 8 its safety guarantees: **any IntLLM kernel-path code that violates the matrix fails to compile, before a kernel binary is ever produced.**

The bottom two rows are particularly load-bearing: a `@kernel` function calling a `@device` function would cross a deployment-domain boundary (kernel-resident code suddenly needing tensor runtime support), and conversely a `@device` function calling a `@kernel` function would invert the privileged/unprivileged direction. Both are mechanically rejected.

Table 11: Fajar Lang context-operation matrix. `OK` = allowed, `ERROR XXNNN` = rejected with the indicated error code. The `@safe` column is the most restrictive subset; `@unsafe` allows all operations; `@kernel` and `@device` are the two specialized contexts that the IntLLM kernel-path (Section 8) and the training-time `@device` tensor operations (Section 4) live in respectively. Source: CLAUDE.md §5.3, ERROR_CODES.md §5.

Operation	`@safe`	`@kernel`	`@device`	`@unsafe`
`let x = 42`	OK	OK	OK	OK
`String::new()`	OK	KE001	OK	OK
`zeros(3,4)` / `relu()`	TE	KE002	OK	OK
`alloc!(4096)`	ERROR	OK	DE002	OK
`*mut T` dereference	ERROR	OK	DE001	OK
`irq_register!()`	ERROR	OK	DE002	OK
Call `@device fn`	OK	KE003	OK	OK
Call `@kernel fn`	OK	OK	DE002	OK

9.2 Compile-time enforcement — error catalog

Table 12 reproduces the kernel- and device-context error codes that the Fajar Lang compiler emits when the matrix of Table 11 is violated. Each error code corresponds to a specific check in the analyzer pass between the parser and the code generator, runs at compile time, and is a hard error (not a warning).

A representative compile-time output for KE001 (HeapAllocInKernel):

```
error[KE001]: heap allocation not allowed
              in @kernel context
  --> kernel.fj:8:5
   |
7  | @kernel
8  | fn init_kernel() {
9  |     let s = String::new();
   |             ^^^^^^^^^^^^^ heap alloc
   |
   = note: @kernel operates without heap
   = help: use stack-allocated arrays or
           alloc!() for raw memory
```

The error includes the source-span pointer to the offending call site, a note explaining the kernel-context invariant being violated, and a help suggestion pointing at the kernel-safe alternative (`alloc!()` for raw memory). All seven error codes follow the

Table 12: Kernel and Device context error codes from `docs/ERROR_CODES.md` §5.1-5.2. Total: 4 kernel errors (KE001-KE004) + 3 device errors (DE001-DE003).

Code	Name	Trigger example
KE001	HeapAllocInKernel	`String::new()`, `Vec::new()` in `@kernel`
KE002	TensorInKernel	`zeros()`, `relu()`, `.backward()` in `@kernel`
KE003	DeviceCallInKernel	`@device fn` called from `@kernel`
KE004	InvalidKernelOp	Operations requiring heap (e.g. `format!()`)
DE001	RawPointerInDevice	`*mut T` dereference outside `@unsafe` block
DE002	HardwareInDevice	`irq_register!`, `port_write!`, `map_page!`
DE003	InvalidDeviceOp	`asm!()`, raw-memory access in `@device`

*[Figure pending — drafted camera-ready. Pipeline diagram: .fj source → lexer → tokens → parser → AST → analyzer (3 sub-passes: type-check, scope-check, **context-check**) → analyzed AST → codegen (Cranelift / LLVM / interpreter) → kernel binary / userspace binary / interpreted execution. The context-check sub-pass is highlighted as the load-bearing C5 contribution; it operates on the analyzed AST AFTER type-check has attached annotations to every function and call site, and BEFORE codegen, so violations are caught with full type information but without spending codegen effort on rejected programs.]*

Figure 5: Fajar Lang compilation pipeline. The context-check sub-pass of the analyzer is the load-bearing C5 contribution: it traverses the analyzed AST and rejects any call-site edge crossing the context-operation matrix of Table 11.

same source-span + note + help template, satisfying the diagnostic-quality bar of established systems languages [Klein et al., 2009].

9.3 Compilation pipeline

The four-context type system is enforced by the analyzer pass of the Fajar Lang compiler, which sits between parsing and code generation. Figure 5 (deferred to camera-ready) plots the pipeline; the textual description here suffices for the contribution claim.

The context-check is implemented as a tree-walk over the analyzed AST. For each function definition, it records the function's context annotation (or assigns `@safe` as default); for each call site inside that function, it checks whether the called function's context is reachable from the calling function's context per the matrix; for each operation expression, it checks whether the operation is allowed in the calling context. Context violations produce a hard compile error with the source span; the program does not proceed to code generation.

9.4 Bridge pattern for cross-domain code

A common deployment shape combines kernel-side I/O (sensor read, hardware register access) with device-side ML inference; the Fajar Lang `@safe` subset serves as the glue layer that bridges the two without violating either context's invariants:

```
@kernel
fn read_sensor() -> [f32; 4] {
    // hardware-register reads here
    // ...
    return data
}

@device
fn infer(x: Tensor) -> Tensor {
    // tensor ops here
    return model.forward(x)
}

@safe
fn bridge() -> Action {
    let raw = read_sensor()
    let result = infer(
        Tensor::from_slice(raw))
    return Action::from_prediction(result)
}
```

The bridge function is `@safe` because it constructs a `Tensor` (forbidden in `@kernel`) and reads sensor data (the `read_sensor` return type is a stack-allocated array, not a raw pointer, so the data crosses the `@kernel` → `@safe` boundary cleanly). The type system permits `@safe` to call both `@kernel` and `@device` functions because the `@safe` caller itself is not subject to either context's specialized invariants. This bridge pattern is the canonical way to compose kernel-side I/O with device-side ML in Fajar Lang programs; the IntLLM kernel-path of Section 8 uses it for the `model-load` / `ask` shell command flow.

9.5 Honest scope: what the type system does and does not guarantee

What C5 establishes:

1. Compile-time rejection of code that would violate

the kernel-context invariants of Section 8 (no heap, no tensor, no `@device` call). Programs that pass the analyzer are guaranteed to satisfy these invariants *at the language level* for any kernel-path code path.

2. Hard errors (not warnings) — programs with safety violations do not produce kernel binaries. This rules out the failure mode where a documentation-grade convention is silently violated by a future contributor and only discovered at runtime.

3. Source-span + note + help diagnostic quality matching established systems-language standards.

What C5 does NOT establish:

1. Formal-verification-grade correctness of the *implementation* of the analyzer itself. The seL4 kernel [Klein et al., 2009] extends compile-time invariants with machine-checked proofs that the compiler implementation correctly discharges those invariants; we do the former, not the latter. A sufficiently determined contributor with patch access to the analyzer could in principle disable a check.

2. Correctness of `@unsafe` blocks. The escape hatch (intentional, used at FFI boundaries) explicitly opts out of the type-system enforcement; kernel-binary safety inside `@unsafe` relies on review-and-audit conventions, not type-system guarantees.

3. Runtime-detectable failures (memory corruption, use-after-free) that require dynamic analysis. The type system rules out a class of bugs (the seven error codes of Table 12) but is not a substitute for dynamic safety mechanisms (page-table protection, canary values) that the FajarOS Nova kernel additionally provides.

The C5 contribution sits at a specific point on the safety-mechanism spectrum: stronger than userspace ML deployment's documentation-grade conventions [Ma et al., 2024, Xu et al., 2025], weaker than seL4's machine-checked proofs [Klein et al., 2009]. We claim the latter position explicitly is out of scope; the former is what existing ternary-LLM literature defaults to, and the type-system layer is a material improvement over that default.

10 Discussion and Future Work

10.1 Synthesis of contributions

The five contributions of this paper compose into a single deployment-grade demonstration that is not addressed by any single prior work along the union of relevant axes (Section 5.3, Section 6.3; full multi-axis positioning in `docs/INTLLM_RELATED_WORK.md` §7.1):

- **C1.** The Phase D scaling chain extends 1.58-bit ternary training-from-scratch validation downward from BitNet b1.58's 2 B-parameter regime [Ma et al., 2024] into the 22-74 M-parameter range relevant for kernel-context deployment, with monotonic widening gate margins ($0.118 \rightarrow 0.210 \rightarrow 0.279$ nat) over a $\sim$3.5$\times$ parameter range and a 1.94$\times$ end-to-end PPL reduction.

- **C2.** The Phase E1 bilingual corpus v1.0 (25.67 B tokens at 60:40 ID:EN, 0% synthetic, mechanical-consistency-gated) and the Q5 Mini-scale baseline are the first fully-public bilingual ternary LM resources at the 22 M-parameter scale, reproducible in $\sim$42 minutes on commodity laptop hardware.

- **C3.** The FajarOS Nova v3.9.0 IntLLM kernel-path demonstrates a 1.58-bit ternary LM running entirely inside the operating-system kernel address space, with a 4-invariant CI-gated regression test and binary footprint within the $\leq$ 16 MB total budget at Mini and Base scales (Medium marginally overruns; identifies v9-format compression as follow-up).

- **C4.** Two outlier-handling features ported from the post-hoc-quantization literature (balanced_calib via SmoothQuant [Xiao et al., 2022] / AWQ [Lin et al., 2023]; Hadamard rotation via QuaRot [Ashkboos et al., 2024]) FAIL their mechanical adoption gates by margins of -82.13 and -0.12 nat respectively. Three-cause attributions per failure converge on a common pattern: *post-hoc-literature precedents do not transfer cleanly to training-from-scratch ternary at small scale.*

- **C5.** The Fajar Lang four-context type system enforces the kernel-context invariants of C3 at compile time via the seven KE001-KE004 / DE001-DE003 error codes, sitting at a specific point on the safety-mechanism spectrum stronger than userspace ML deployments' documentation-grade conventions and weaker (explicitly) than seL4-class machine-checked proofs [Klein et al., 2009].

The composition of these five — none of which is individually a breakthrough — is what makes IntLLM a coherent embedded-AI-plus-OS-integration system rather than an incremental advance on any single axis.

10.2 What this paper does NOT claim

The paper is deliberately scoped to what we can mechanically verify on the available infrastructure:

1. **Frontier benchmark performance.** IntLLM is 25-100× smaller than BitNet b1.58 2B4T [Ma et al., 2024] and orders of magnitude smaller than frontier 7 B+ models. We do not claim parity on multi-choice reasoning benchmarks; the 22-74 M scale is below the emergence threshold for those skills per Pythia [Biderman et al., 2023] and Chinchilla [Hoffmann et al., 2022].

2. **Ternary parity with FP16 at the same parameter count.** Pythia-160 M and SmolLM2-135 M [Allal et al., 2024] achieve materially better Wikitext word-PPL at comparable parameter counts. The trade is parameter efficiency for kernel-deployable binary footprint and integer-native arithmetic.

3. **Universal failure of outlier-handling features.** C4's two negative results are at our specific regime (Mini-scale, training-from-scratch, bilingual data); the literature's positive post-hoc results at $\geq$ 7 B FP16 are not refuted, and Phase F.6 explicitly tests whether the post-hoc protocol recovers the literature result on our trained Mini checkpoint.

4. **Production-grade FajarOS Nova.** The kernel passes its own 35-test regression suite but is research-grade; no formal verification, no hardened resource isolation beyond what the page-table mechanisms provide, no defense-in-depth against malicious inputs.

5. **Bare-metal performance numbers.** Section 8.4 explicitly tags the tok/s and watts entries of Table 10 as PEND, deferred to a future deployment-platform partner.

These delimitations are not modesty — they are research-integrity requirements per the methodology of Section 4.5 (R3, R6).

10.3 Phase F: immediate-next research agenda

The two C4 negative results, plus the three outlier-handling features cut from the paper's scope under the Path A decision (Section 7.5), define a five-entry research agenda committed in `FJQ_PHASE_F_TAX_VERTICAL_ROADMAP.md` v1.3 §4.1:

- **F.5 Post-training PTQ-style activation calibration.** Replace the all-time-max running-max accumulator with a SmoothQuant [Xiao et al., 2022]-style held-out post-training calibration on a fixed batch from the Q5-trained Mini checkpoint. Tests whether the balanced_calib failure was about WHEN calibration happens (during-training accumulator) rather than WHAT calibration is computed (per-channel scale). Cheapest of the five; entry condition is sub-day smoke showing $\geq +10\%$ MSE reduction on outlier channels.

- **F.6 Post-hoc QuaRot on Q5-trained Mini checkpoint.** Apply the canonical QuaRot [Ashkboos et al., 2024] recipe (load checkpoint, fuse Hadamard into adjacent weights via algebraic identity, re-evaluate without re-training) to the Q5-trained checkpoint. Tests whether the Hadamard failure was about training-from-scratch vs. post-hoc, controlling for HGRN-Bit architecture. Hard prerequisite F.6.1: pre-flight outlier-concentration measurement on `o_proj` inputs (the hindsight measurement Section 7.3 cause 2 should have run before E2.1).

- **F.7 FP8 (E4M3) mixed-precision for `lm_head` + `attn.W_o`.** Original Phase E2.2 sub-feature, cut to Phase F under Path A. Tests whether structurally-different (mixed-precision rather than outlier-handling) features behave differently than F.5/F.6/E2.4/E2.1. Hardware compatibility check: `torch.float8_e4m3fn` availability on RTX 4090 Laptop pinned PyTorch version is the F.7.0 gate before F.7.1+ commits resources.

- **F.8 FP16 distillation during QAT.** Original Phase E2.3, cut to Phase F. Teacher = Phase D Medium FP32 master (per Q3 closure, viable via `model.half()`); negative-transfer risk on ID-side samples (English-only teacher, bilingual student) addressed by sampling-aware distill-weight design.

- **F.9 Language-conditioned design.** Original Phase E2.5, cut to Phase F. Re-evaluate Option (a)/(b)/(c) selection given F.5-F.8 empirical evidence; if one of F.5-F.8 demonstrates that Phase E-class features can help under our regime, F.9 carries the explicit responsibility to close the 1.77× → ≤1.5× coherence-ratio headroom the Q5 baseline left open.

The five-entry agenda is paced for V32-cycle execution post-this-paper-acceptance; estimated effort is 4-6 weeks human + ∼10-15 h GPU per sub-feature.

10.4 Longer-term roadmap

Beyond the immediate Phase F entries, three longer-horizon items are acknowledged but explicitly out-of-scope for this paper:

- **Phase E3 bilingual scale-up to Base and Medium.** Q5 is a Mini-scale single-run baseline (Section 6.2); the full bilingual scaling chain (Phase D's Mini → Base → Medium on the bilingual corpus rather than SlimPajama) would multiply the 24K-step Mini budget by ∼5× for Base and ∼10× for Medium. Estimated ∼30-50 hours GPU on RTX 4090 Laptop; deferred until Path A paper acceptance frees calendar.
- **Phase D Stretch training (15 B tokens).** A fourth data point on the Phase D scaling-law curve. The ∼17-day continuous-GPU budget at the laptop's throughput is the binding constraint; nice-to-have but not required for the C1 scaling-law claim. Path A explicitly defers this in `FJQ_PHASE_D_PRODUCTION_PLAN.md` v1.10 §2.3.
- **Tier 3 Phase F.1-F.4 tax-vertical TaxPrime fine-tuning.** The original Phase E plan included a tax-domain Indonesian fine-tune as a third deliverable (alongside Tier 1 kernel-context and Tier 2 bilingual base); the Path A scope decision deferred this entirely to Phase F under `FJQ_PHASE_F_TAX_VERTICAL_ROADMAP.md` v1.3 §F.1-F.4. Founder-bandwidth + NDA-pending TaxPrime archive ingest are the binding constraints; Phase F activation gates on this paper's acceptance.

10.5 Methodological reflection: Path A as a methodology contribution

The decision to STOP Phase E2 after two ablations both FAILED, rather than pushing through three more ablations to fill out the original five-feature combined-ablation table, is itself a methodology choice that this paper claims as part of contribution C4. The reasoning chain that motivated the cut, committed in `FJQ_PHASE_E_PATH_A_PAPER_OUTLINE.md` v1.0:

1. **The 0/2 prior result is empirical evidence about the regime, not just about the two specific features.** Both failures attributed to the same root cause (post-hoc-literature features applied to training-from-scratch); the prior probability that features structurally similar to those (F.7 FP8, F.8 distillation, F.9 lang-cond — all originally Phase E ablation candidates) succeed under the same regime is therefore lower than the original Phase E plan v1.0 assumed.
2. **Continuing under that updated prior costs ∼3 weeks and ∼3-5 hours of GPU per ablation, against an expected paper yield of ≤ 1 row likely to PASS.** The marginal evidence per additional ablation is low; the marginal cost is high.
3. **The negative-results paper is publishable as written.** Two clean failures with three-cause attributions and four future-work pointers (F.5-F.8) is enough material for a contribution-C4 narrative that the next research team can build on; running three more ablations adds bulk but not new structural insight.
4. **Cutting losses redirects the same calendar to paper LaTeX writing,** which Path A's outline shows is the binding path to arXiv-and-MLSys submission.

We treat this as a successful Phase E2 outcome rather than a failure of the original plan: the methodology of mechanical gates committed before experiments ran (R6) and honest reporting of negative results (R3) is what allowed the cut decision to be made on empirical evidence rather than sunk-cost reasoning. The kind of paper that emerges — honest negative results plus deployable positive contributions — is the kind of paper the field needs more of, not fewer of, at the small-scale ternary intersection where literature is currently sparse.

References

Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. SmolLM2: When smol goes big — a suite of small language models. HuggingFace TB technical report; checkpoint family `HuggingFaceTB/SmolLM2-*`, 2024.

Anonymous. Calibrating beyond english: Language diversity for better quantized multilingual LLMs. *arXiv preprint arXiv:2601.18306*, 2026. Citation deferred to camera-ready for verified author list.

Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *arXiv preprint arXiv:2404.00456*, 2024.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. *arXiv preprint arXiv:2304.01373*, 2023. ICML 2023.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. NeurIPS 2022 (Chinchilla).

Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. SpinQuant: LLM quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024. Companion 2 B parameter Microsoft release: `microsoft/bitnet-b1.58-2B-4T`.

Alexei Starovoitov et al. eBPF: Extended Berkeley packet filter for Linux kernel-side programmability. Linux kernel mainline; documentation at `https://www.kernel.org/doc/html/latest/bpf/`, 2014. Cited as the canonical reference for kernel-side bytecode execution; eBPF-AI line of work derives from this primitive.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaiyu Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.

Jinheng Xu, Hongyu Wang, Shuming Ma, and Furu Wei. Bitnet.cpp: Efficient edge inference for ternary LLMs. *arXiv preprint arXiv:2502.11880*, 2025.

Rui-Jie Zhu, Yu Zhang, Ethan Sifferman, Tyler Sheaves, Yiqiao Wang, Dustin Richmond, Peng Zhou, and Jason K. Eshraghian. Scalable MatMul-free language modeling. *arXiv preprint arXiv:2406.02528*, 2024. NeurIPS 2024.